

## Lect(14-16)

# Function in JavaScript:

A function is a block of reusable code designed to perform a particular task. You can define it once and use it multiple times.

### Syntax:

```
function functionName(parameter1, parameter2) {  
    return result;  
}
```

### Example:

```
function greet(name) {  
    return "Hello, " + name + "!";  
}  
console.log(greet("John"));
```

## Function Declaration vs Function Expression

A Function Declaration defines a function using the `function` keyword at the top level.

A Function Expression assigns a function to a variable (or constant).

It can be named or anonymous.

| Type                 | Example                                   | Hoisting |
|----------------------|-------------------------------------------|----------|
| Function Declaration | <code>function sayHi() {}</code>          | Yes      |
| Function Expression  | <code>const sayHi = function() {};</code> | No       |

### Example:

```
// Function Declaration
function add(a, b) {
  return a + b;
}

// Function Expression
const multiply = function(a, b) {
  return a * b;
};
```

## Arrow Functions (ES6)

Simpler syntax for writing small functions.

### Syntax:

```
const add = (a, b) => a + b;
```

### Example:

```
const greet = name => `Hello ${name}`;
console.log(greet("John"));
```

**Note:** Arrow functions don't have their own **this** or **arguments**.

## Anonymous Functions

Functions without a name — often used in event handlers or callbacks.

```
setTimeout(function() {
  console.log("Executed after 2 seconds");
}, 2000);
```

# Immediately Invoked Function Expression (IIFE)

Executes immediately after it's defined.

```
(function() {  
  console.log("IIFE executed immediately!");  
})();
```

**Use Case:** To avoid global scope pollution.

## Higher-Order Functions

A Higher-Order Function (HOF) takes another function as an argument or returns one.

**Syntax:**

```
function greetUser(greetFn, name) {  
  return greetFn(name);  
}  
function hello(name) {  
  return "Hello " + name;  
}  
console.log(greetUser(hello, "John"));
```

**Example:**

```
const numbers = [1, 2, 3];  
const squares = numbers.map(num => num * num);  
console.log(squares);
```

## Callback Functions

A function passed as an argument to another function.

```
function processUserInput(callback) {  
  const name = "John";  
  callback(name);  
}  
processUserInput(function(name) {  
  console.log("Hello " + name);  
});
```

# Closures

A closure gives access to an outer function's scope from an inner function.

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    return count;  
  };  
}  
const counter = outer();  
console.log(counter());  
console.log(counter());
```

**Use Case:** Data privacy and encapsulation.

# Default Parameters

Provide default values for parameters.

```
function greet(name = "Guest") {  
  console.log("Hello " + name);  
}  
greet();  
greet("John");
```

# Rest Parameters

Collects all remaining arguments into an array.

```
function sum(...numbers) {  
  return numbers.reduce((a, b) => a + b, 0);  
}  
console.log(sum(10, 20, 30));
```

# Function Currying

Breaking a function with multiple arguments into nested single-argument functions.

```
function add(a) {  
  return function(b) {  
    return a + b;  
  };  
}  
console.log(add(5)(10));
```

# Function Chaining

Function chaining means calling multiple methods on the same object in a single statement, one after another.

Each method must return the object itself (**this**) to allow chaining.

```
class Calculator {  
  constructor(value = 0) {  
    this.value = value;  
  }  
  add(num) {  
    this.value += num;  
    return this; // return the current object  
  }  
  multiply(num) {  
    this.value *= num;  
    return this;  
  }  
  print() {  
    console.log(this.value);  
    return this;  
  }  
}  
// Function chaining  
const result = new Calculator(10)  
  .add(5)  
  .multiply(4)  
  .print();
```

## Function Constructor

You can create a function using the **Function** constructor (rarely used).

```
const add = new Function("a", "b", "return a + b;");
console.log(add(2, 3));
```

## Prototype and Constructor Functions

### What is Prototype in JavaScript?

In JavaScript, the prototype is a container for methods and properties. When methods or properties are added to the prototype of a function, array, or string, they become shared by all instances of that particular object type. Prototype in general can be understood as a mould and all its instances can be considered as the car made from it.

```
Object.prototype.print = function () {
    console.log('I am from object prototype')
}
let b = {
    name: 'Pranjal',
    age: 21
}
b.print()
```

- Every JavaScript object has a prototype, which contains shared methods and properties that all instances of that object can use. It's like a template for objects.
- When you add a method or property to an object's prototype, all instances of that object automatically have access to it.
- In this example, the print() method is added to **Object.prototype**, so any object (like b) can use it.
- Using prototypes helps you avoid repeating code. Instead of defining the same methods for each object, you define them once in the prototype, and all objects can use them. This makes your code more efficient.

## How Prototype Works in JavaScript?

- In JavaScript, each object has an internal `[[Prototype]]` property, which points to another object. This allows the object to inherit properties and methods from its prototype.
- When you access a property or method on an object, JavaScript first checks the object itself. If it's not found, it looks in the object's prototype and continues up the prototype chain until it either finds the property or reaches null.
- Functions in JavaScript have a prototype property. This is where you add properties or methods that you want to be available to all instances of objects created by that constructor function.
- Objects created using a constructor function inherit properties and methods from the constructor's prototype. This allows for reusable code and shared behaviour across multiple objects.
- You can add new properties or methods to an object's prototype, and all instances of that object will automatically have access to the new functionality. This is a common way to extend built-in objects like Array or Object.

## Creating Constructor Functions

A Constructor in JavaScript is a special function used to create and initialize objects, typically using the `new` keyword. It sets up properties and methods on the newly created object, allowing for the creation of multiple instances with shared behavior.

```
function Person(name) {  
    this.name = name;  
}  
Person.prototype.sayHello = function () {  
    console.log(`Hello, my name is ${this.name}.`);  
};  
const n = new Person("Rahul");  
n.sayHello();
```

- A constructor function is a special function used to create and initialize new objects. In this case, Person is a constructor that takes a name argument and assigns it to the name property of the new object using this.
- The sayHello method is added to the Person prototype, meaning all instances of Person will have access to this method, allowing them to say "Hello" with their name.
- The expression new Person("Rahul") creates a new object using the Person constructor, passing "Rahul" as the name argument. The new object is stored in the name variable.
- The sayHello() method is then called on the name object, which outputs "Hello, my name is Rahul." to the console.

## Adding Methods to Prototype

Instead of defining methods inside the constructor, we define them on the prototype for efficiency.

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}  
Person.prototype.greet = function() {  
  console.log("Hello, my name is " + this.name);  
};  
const p1 = new Person("John", 22);  
p1.greet();
```

## Why use prototype?

### 1. Memory Efficiency

If you define methods inside a constructor function, every new object you create gets its own copy of each method — wasting memory.

By adding methods to the prototype, all instances share a single copy of those methods



```
function Person(name) {
  this.name = name;
}

// Method added to prototype (shared by all objects)
Person.prototype.greet = function() {
  console.log("Hello, I'm " + this.name);
};

const p1 = new Person("Amit");
const p2 = new Person("Riya");

console.log(p1.greet === p2.greet); // true → shared method
```

## 2. Code Reusability

You can define reusable methods or properties once on the prototype, and all instances can automatically access them — no need to rewrite the same code.

```
Person.prototype.sayBye = function() {
  console.log(this.name + " says goodbye!");
};
p1.sayBye();
p2.sayBye();
```

## 3. Easy to Extend or Modify

You can add new methods to prototypes even after creating objects — and all existing instances will instantly have access.

```
Person.prototype.walk = function() {
  console.log(this.name + " is walking...");
};

p1.walk(); // works even though p1 was created earlier
```

## 4. Supports Inheritance (Prototype Chain)

Prototypes make it possible to inherit properties and methods between objects (using the prototype chain).

This allows one constructor to build upon another:

```
function Student(name, grade) {
  Person.call(this, name);
  this.grade = grade;
}

Student.prototype = Object.create(Person.prototype);
Student.prototype.constructor = Student;

Student.prototype.study = function() {
  console.log(this.name + " is studying in grade " + this.grade);
};

const s1 = new Student("Neha", "B.Tech");
s1.greet(); // inherited from Person prototype
s1.study(); // defined in Student prototype
```

## Prototype Chain

In JavaScript, functions, arrays, and strings are all considered types of objects. They are just specialized forms of the general object type. The Object type acts as the parent or base for these different object types. All the other object types like array's, functions and Strings are connected as a child to the Object through prototype.

```
let o = {
  name: "Pranjal",
  age: 21
}
let a = [1, 2, 3, 4, 5]
let s = "Hello World"
function p() {
  console.log('My name is xyzabc')
}
Object.prototype.common = function () {
  console.log('I am a shared method from prototype')
}
o.common()
a.common()
s.common()
p.common()
```

## Output :

```
I am a shared method from prototype  
I am a shared method from prototype  
I am a shared method from prototype  
I am a shared method from prototype
```

## Inheritance using Prototype

Inheritance using Prototype in JavaScript is a mechanism where one object can access the properties and methods of another object through a prototype chain.

In other words, objects can inherit features (methods or properties) from other objects using the prototype property of constructor functions.

Every function in JavaScript has a **prototype** object, and when you create an object using that function (with **new**), the new object's internal **[[Prototype]]** (or **\_\_proto\_\_**) is linked to that constructor's **prototype**.

## Example:

```
function Animal(name) {  
  this.name = name;  
}  
Animal.prototype.eat = function() {  
  console.log(`${this.name} is eating`);  
};  
function Dog(name, breed) {  
  Animal.call(this, name);  
  this.breed = breed;  
}  
Dog.prototype = Object.create(Animal.prototype);  
Dog.prototype.bark = function() {  
  console.log(`${this.name} is barking`);  
};  
const dog1 = new Dog("Tommy", "Labrador");  
dog1.eat();  
dog1.bark();
```

## ES6 Classes (Syntactic Sugar over Prototypes)

ES6 Classes in JavaScript are a **syntactic sugar** (simplified, cleaner syntax) over JavaScript's existing prototype-based inheritance system.

They provide a more familiar, structured, and object-oriented way to create objects and handle inheritance — but under the hood, classes still use prototypes to link objects and share methods.

### Example:

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  greet() {  
    console.log(`Hi, I am ${this.name}`);  
  }  
}  
  
const p = new Person("John", 22);  
p.greet();
```

Under the hood, classes use prototype-based inheritance.

### Static Methods

Static methods in JavaScript are methods that belong to the class itself, not to the instances of that class.

They are defined using the **static** keyword inside a class and can be called directly using the class name, without creating an object.

## Example:

```
class MathUtils {  
  static add(a, b) {  
    return a + b;  
  }  
}  
console.log(MathUtils.add(5, 3));
```

## Important Notes

| Topic                | Key Point                       |
|----------------------|---------------------------------|
| Function Declaration | Hoisted                         |
| Arrow Function       | No own <b>this</b>              |
| Closure              | Inner function + outer variable |
| Prototype            | Used for inheritance            |
| Constructor Function | Used to create similar objects  |
| Class                | Syntactic sugar for prototype   |

## Comparison: Types of Functions in JavaScript

| Function Type                                         | Syntax Example                                                                        | Hoisted | Has its own this?         | Can be Anonymous? |
|-------------------------------------------------------|---------------------------------------------------------------------------------------|---------|---------------------------|-------------------|
| <b>Function Declaration</b>                           | <code>js function greet()<br/>{ return "Hi!"; }</code>                                | Yes     | Yes                       | No                |
| <b>Function Expression</b>                            | <code>js const greet =<br/>function() { return<br/>"Hi!"; };</code>                   | No      | Yes                       | Yes               |
| <b>Arrow Function (ES6)</b>                           | <code>js const greet =<br/>name =&gt; `Hi<br/>\${name}`;</code>                       | No      | No (inherits from parent) | Yes               |
| <b>Anonymous Function</b>                             | <code>js<br/>setTimeout(function(<br/>) {<br/>console.log("Hi");<br/>}, 1000);</code> | No      | Yes                       | Always            |
| <b>Immediately Invoked Function Expression (IIFE)</b> | <code>js (function() {<br/>console.log("Runs<br/>instantly"); })();</code>            | No      | Yes                       | Yes               |
| <b>Higher-Order Function</b>                          | <code>js function<br/>apply(fn) { fn(); }</code>                                      | N/A     | Depends                   | Yes               |
| <b>Callback Function</b>                              | <code>js doSomething(() =&gt;<br/>console.log("Done"))<br/>;</code>                   | N/A     | Depends                   | Yes               |
| <b>Constructor Function</b>                           | <code>js function<br/>Person(name){<br/>this.name = name; }</code>                    | No      | Yes                       | No                |
| <b>Class Method (ES6)</b>                             | <code>js class A { greet()<br/>{ return "Hi"; } }</code>                              | No      | Yes                       | No                |

## Scenario-Based Problem:- Online Learning System

```
function greetUser(name) {
  return `Welcome, ${name}, to LearnHub!`;
}
console.log(greetUser("John"));
const calculateTotalPrice = function (price, taxRate = 0.18) {
  return price + price * taxRate;
};
console.log("Total Price:", calculateTotalPrice(1000));
const courseSummary = (name, price) => `Course: ${name}, Price: ₹${price}`;
console.log(courseSummary("JavaScript Mastery", 1500));
function applyDiscount(courses, discountFn) {
  return courses.map(course => discountFn(course));
}
const discountFn = course => ({
  ...course,
  price: course.price * 0.9
});
const courses = [
  { name: "ReactJS", price: 2000 },
  { name: "NodeJS", price: 2500 }
];
const discounted = applyDiscount(courses, discountFn);
console.log("Discounted Courses:", discounted);
function enrollmentCounter() {
  let count = 0;
  return function () {
    count++;
    console.log(`Total Enrollments: ${count}`);
  };
}
const enroll = enrollmentCounter();
enroll();
enroll();
enroll();
function Student(name, course) {
  this.name = name;
  this.course = course;
}
const s1 = new Student("John", "ReactJS");
console.log("Student:", s1);
```

```

Student.prototype.displayDetails = function () {
  console.log(`${this.name} enrolled in ${this.course}`);
};
s1.displayDetails();
function PremiumStudent(name, course, membership) {
  Student.call(this, name, course);
  this.membership = membership;
}
PremiumStudent.prototype = Object.create(Student.prototype);
PremiumStudent.prototype.constructor = PremiumStudent;
PremiumStudent.prototype.getBenefits = function () {
  console.log(`${this.name} has ${this.membership} membership benefits!`);
};
const p1 = new PremiumStudent("Riya", "NodeJS", "Gold");
p1.displayDetails();
p1.getBenefits();
class Course {
  constructor(title, duration, price) {
    this.title = title;
    this.duration = duration;
    this.price = price;
  }
  showCourse() {
    console.log(`${this.title} (${this.duration}) - ₹${this.price}`);
  }
}
class OnlineCourse extends Course {
  constructor(title, duration, price, instructor) {
    super(title, duration, price);
    this.instructor = instructor;
  }
  courseInfo() {
    console.log(`Instructor: ${this.instructor}`);
  }
}
const jsCourse = new OnlineCourse("JavaScript Mastery", "6 weeks", 2999,
"Ram Singh");
jsCourse.showCourse();
jsCourse.courseInfo();
class CourseUtil {
  static averagePrice(courseArray) {
    const total = courseArray.reduce((sum, c) => sum + c.price, 0);
    return total / courseArray.length;
  }
}

```



```
    }  
  }  
  const courseList = [  
    { title: "React", price: 2000 },  
    { title: "Node", price: 2500 },  
    { title: "MongoDB", price: 1500 }  
  ];  
  console.log("Average Course Price:", CourseUtil.averagePrice(courseList));  
  console.log("\n Summary ");  
  console.log("Topics Covered: Function Declaration, Expression, Arrow  
Functions, HOF, Closure, Constructor, Prototype, Inheritance, Classes,  
Static Methods.");  
  console.log("Scenario: Online Learning System Simulation Completed  
Successfully!");
```